

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Отчёт
По лабораторной работе №5
по дисциплине Компьютерная графика
Вариант: -

Работу выполнили:
Молчанов Федор Денисович Р3313
Метель Леонард Валерьевич Р3312
Кравченко Дмитрий Олегович Р3319
Работу принял:
Жданов Андрей Дмитриевич

г. Санкт-Петербург 2026

Лабораторная работа N 5

Фильтрация синтезированного изображения

1. Цель работы

Цель лабораторной работы – применить фильтры постобработки к изображению, полученному методом трассировки путей, чтобы уменьшить шумы Монте-Карло. Реализация выполнена в файле `lr5_filters.py`. Входными данными служит `.prz`-архив AOV-буферов, сохраненный из LР4, а результаты фильтрации сохраняются в `.ppm`.

2. Особенность шума синтезированного изображения

Шум в рендере path tracing возникает из-за стохастической оценки интеграла освещенности. Для пикселя используется конечное число сэмплов:

$$\hat{L} = \frac{1}{S} \sum_{s=1}^S L_s$$

При малом S дисперсия оценки остается заметной, особенно в косвенном освещении. В отличие от обычной фотографии, где фильтр обычно видит только RGB-пиксели, в синтезированном изображении доступны дополнительные данные о сцене: глубина, нормали и идентификаторы объектов. Поэтому фильтрация может учитывать не только близость цветов, но и реальные геометрические границы.

3. Входные данные

Функция `load_aov` загружает следующие буферы:

- `direct` – прямое освещение;
- `secondary` – вторичное освещение, где сосредоточена основная часть шума;
- `depth` – глубина первого пересечения;
- `normal` – нормаль поверхности;
- `object_id` – идентификатор объекта;
- `hdr` – исходный суммарный HDR-кадр, если он присутствует.

Фильтры применяются к компоненте `secondary`, после чего итоговый кадр собирается по формуле:

$$I_{\text{result}} = I_{\text{direct}} + F(I_{\text{secondary}})$$

Такой подход сохраняет уже устойчивую часть прямого освещения и подавляет шум преимущественно там, где он наиболее выражен.

4. Гауссова фильтрация

Гауссов фильтр использует только расстояние между пикселями. Для соседнего пикселя (i, j) вес равен:

$$w_{s(i,j)} = \exp\left(-\frac{(x-i)^2 + (y-j)^2}{2\sigma_s^2}\right)$$

Итоговое значение пикселя вычисляется как нормированное среднее:

$$I_{\text{new}}(x, y) = \frac{\sum_{i,j \in \Omega_r} w_s(i, j) I(i, j)}{\sum_{i,j \in \Omega_r} w_s(i, j)}$$

В программе этот метод реализован функциями `make_spatial_kernel`, `_gaussian_row` и `gaussian_blur_rgb`. Его преимущество – простота и устойчивое подавление мелкого шума. Недостаток – размытие границ объектов и мелких деталей, потому что фильтр не различает разные поверхности.

5. Билатеральная фильтрация

Билатеральный фильтр добавляет цветовой вес:

$$w_c(i, j) = \exp\left(-\frac{\text{norm}(I(x, y) - I(i, j))^2}{2\sigma_c^2}\right)$$

Полный вес без учета AOV имеет вид:

$$W = w_s w_c$$

В `lr5_filters.py` билатеральный режим реализован через `edge_aware_filter` с передачей `object_id`. Маска объекта запрещает смешивать пиксели разных объектов:

$$M_{\text{obj}} = 1, \quad M_{\text{obj}} = 0$$

Значение $M_{\text{obj}} = 1$ используется для пикселей одного объекта, а $M_{\text{obj}} = 0$ – для разных объектов.

Тогда итоговый вес:

$$W = w_s w_c M_{\text{obj}}$$

Это уменьшает перетекание цвета через границы объектов, однако цветовой вес может ошибаться на сильно зашумленных участках: случайный шум увеличивает разницу RGB и мешает усреднению похожих точек.

6. Мультилатеральная фильтрация

Мультилатеральный фильтр расширяет билатеральный подход за счет AOV-буферов. Помимо пространственного и цветового веса используются глубина и нормаль:

$$w_z = \exp\left(-\frac{(z(x, y) - z(i, j))^2}{2\sigma_z^2}\right)$$

$$w_n = \exp\left(-\frac{(1 - n(x, y) \cdot n(i, j))^2}{2\sigma_n^2}\right)$$

Полный вес соседа:

$$W = w_s w_c w_z w_n M_{\text{obj}}$$

Глубина помогает не смешивать передние и задние поверхности, а нормаль сохраняет ребра и резкие изменения ориентации. Поэтому мультилатеральная фильтрация лучше подходит

для синтезированных изображений, чем обычное размытие: она использует информацию, которой нет в фотографическом RGB-кадре.

7. Медианная фильтрация

Медианный фильтр заменяет пиксель медианой значений в окрестности:

$$I_{\text{new}}(x, y) = \text{median}(I(i, j))$$

В реализации `object_aware_median_filter` медиана считается только по пикселям с тем же `object_id`. Это делает метод устойчивым к одиночным ярким выбросам, которые часто возникают в `path tracing` как «светлячки». После медианной фильтрации применяется `normalize_median_per_object`: для каждого объекта восстанавливается суммарная энергия вторичного освещения,

$$I_{\text{new obj}} = I_{\text{obj}} \frac{\sum I_{\text{secondary}}}{\sum I_{\text{new secondary}}}$$

Эта нормировка нужна, потому что медиана может уменьшить общую яркость объекта.

8. Приведение HDR к 0..255

Результаты фильтрации остаются HDR-изображениями. Для записи в PPM используется `tonemap_and_gamma`. В режиме `r99` изображение нормируется не по абсолютному максимуму, а по 99-му процентилю:

$$I_{\text{norm}} = \frac{I}{P_{99}}(I)$$

Такой способ менее чувствителен к единичным ярким выбросам. После нормировки значения обрезаются в диапазон $[0, 1]$ и применяется гамма-коррекция:

$$V_{\text{corr}} = V^{\frac{1}{\gamma}}, \quad \gamma = 2.2$$

Затем значения переводятся в 8-битный диапазон:

$$\text{RGB}_8 = \text{round}(255 V_{\text{corr}})$$

Программа также выводит `clip_ratio_before_clamp`, то есть долю значений, которые вышли за допустимый диапазон до обрезки. Этот показатель помогает оценить потерю деталей в светах.

9. Результаты

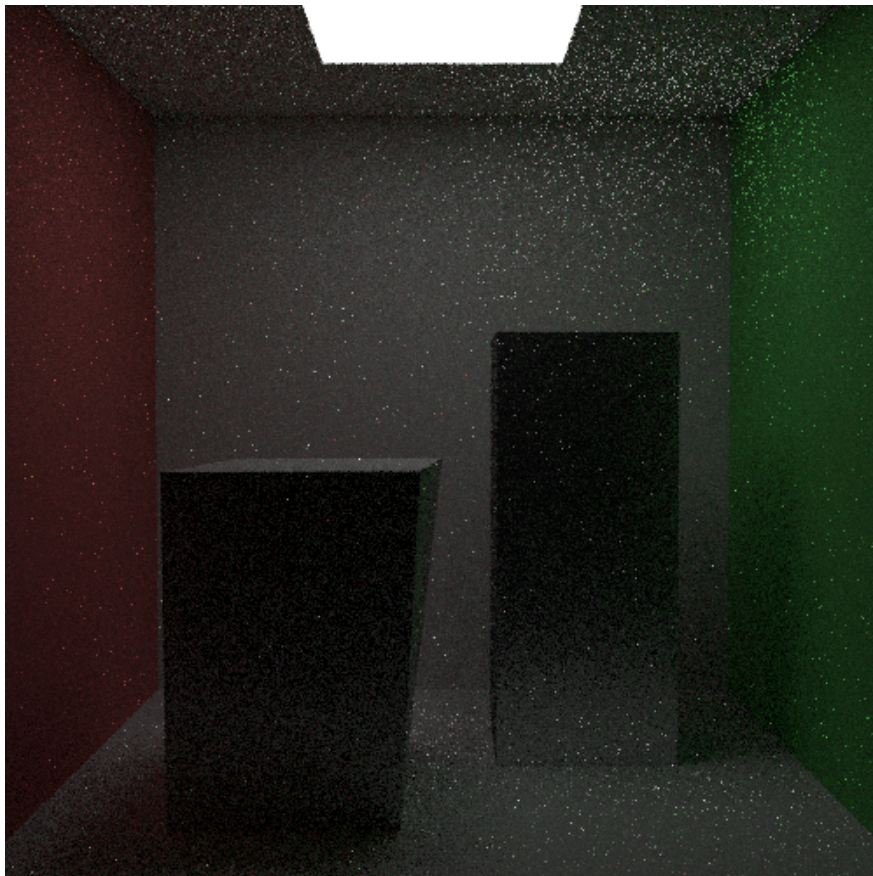


Рис. 1. Исходный зашумленный результат. Исходный файл: renders/lr5/new_render_raw.ppm.

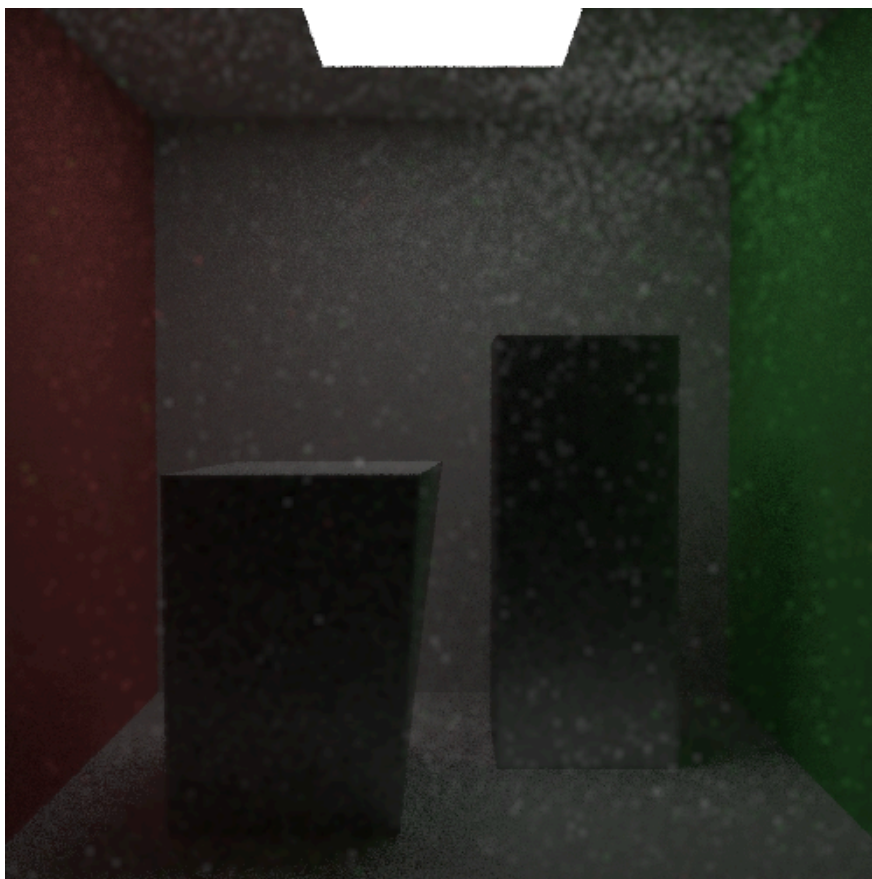


Рис. 2. Результат гауссовой фильтрации. Исходный файл: renders/lr5/
new_render_gaussian.ppm.

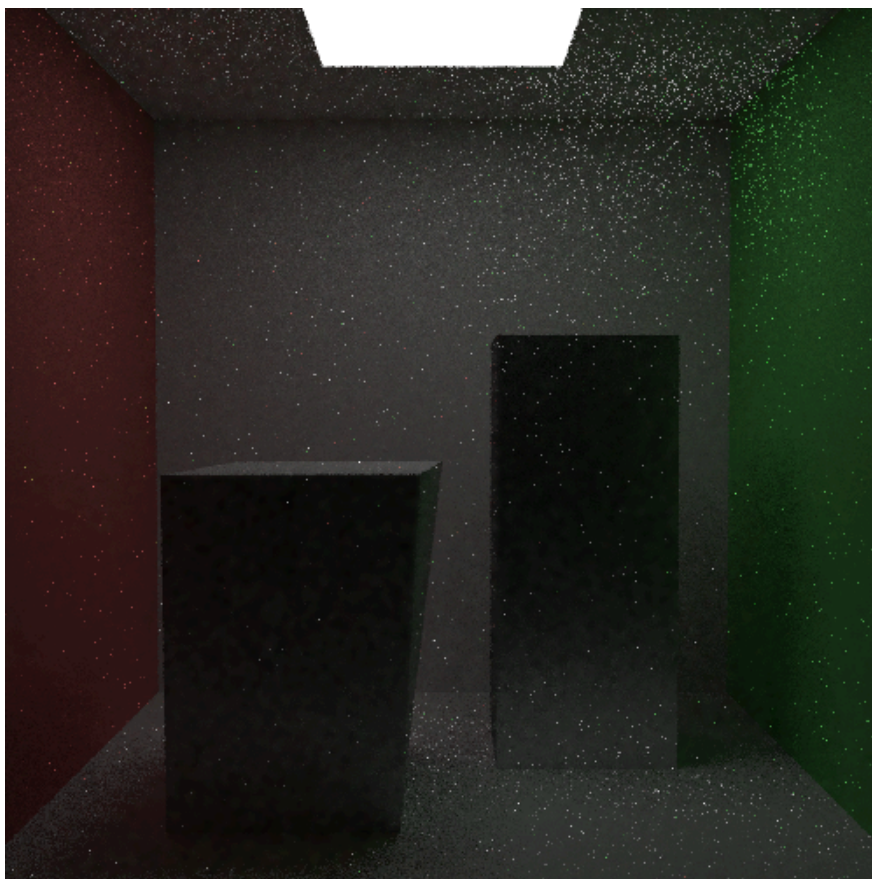


Рис. 3. Результат билатеральной фильтрации. Исходный файл: renders/lr5/
new_render_bilateral.ppm.

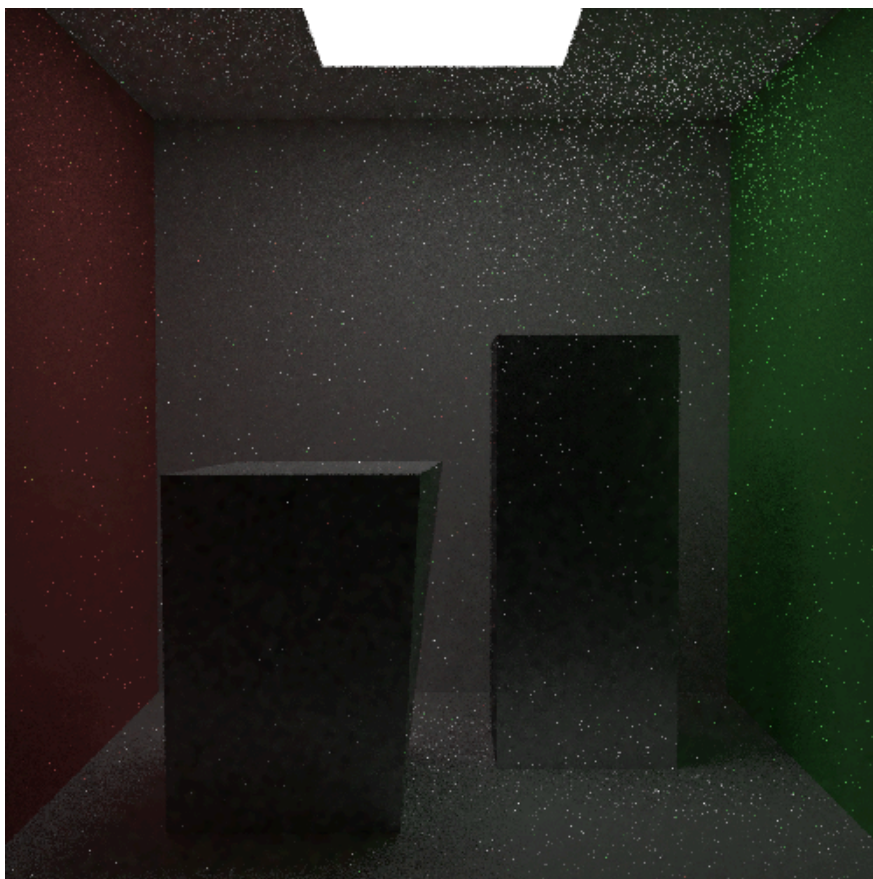


Рис. 4. Результат муьтилатеральной фильтрации с использованием depth, normal и object_id. Исходный файл: renders/lr5/new_render_multilateral.ppm.

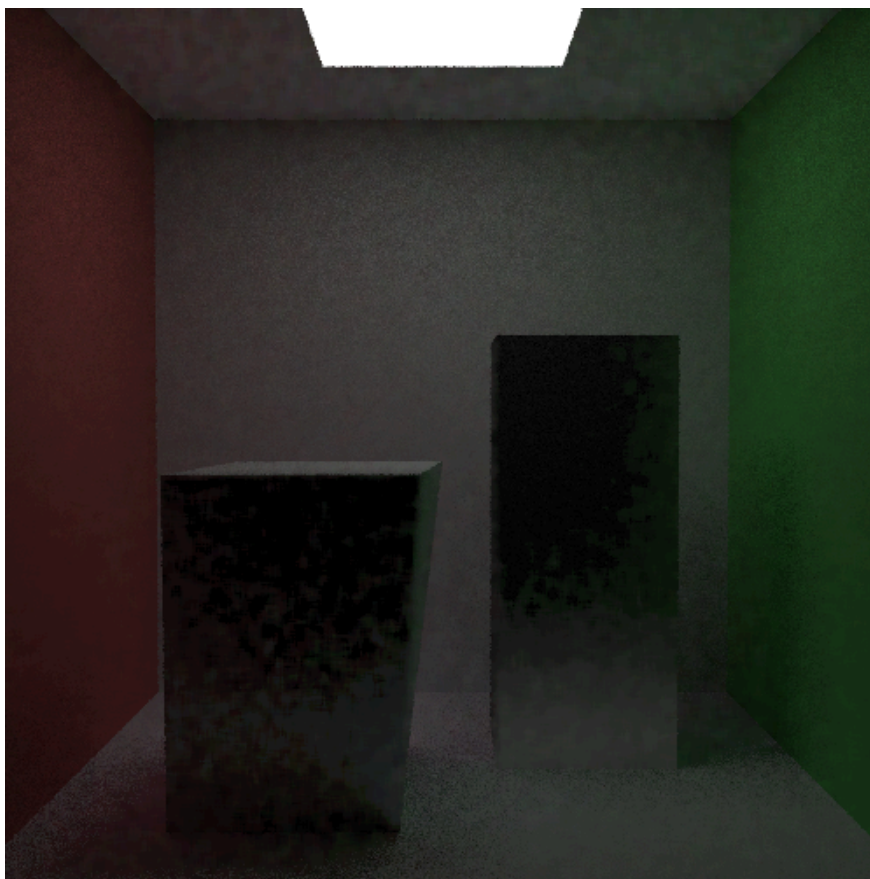


Рис. 5. Результат объектно-ориентированной медианной фильтрации. Исходный файл: renders/lr5/new_render_median.ppm.

На исходном изображении заметен шум Монте-Карло. Гауссов фильтр сглаживает его, но одновременно размывает границы. Билатеральная фильтрация лучше сохраняет контуры за счет цветового веса и `object_id`. Мультилатеральная фильтрация дополнительно использует геометрию сцены, поэтому лучше отделяет реальные границы от случайного шума. Медианный фильтр эффективен против одиночных выбросов, но требует нормировки энергии, чтобы не затемнять объект.

10. Вывод

В работе реализован конвейер фильтрации синтезированного изображения: загрузка AOV-буферов, фильтрация компоненты `secondary`, восстановление итогового кадра, тональная компрессия и сохранение результатов в PPM. Рассмотрены гауссов, билатеральный, мультилатеральный и медианный фильтры. Наиболее обоснованным для path tracing является edge-aware подход с использованием `depth`, `normal` и `object_id`, потому что он подавляет шум, не разрушая геометрические границы сцены.